The Dynamic Sporadic Server Algorithm for Aperiodic Task Scheduling in EDF Systems

Oleg Kelner
Hochschule Hamm-Lippstadt
Lippstadt, Germany
oleg.kelner@stud.hshl.de

***Abstract*—Real-time systems must schedule hard periodic tasks alongside soft aperiodic requests, which demand fast response without endangering periodic deadlines. This paper adapts the fixed-priority Sporadic Server to Earliest Deadline First (EDF) scheduling as the Dynamic Sporadic Server (DSS): we derive its burst-anchored replenishment rule, formalise it as an executable state machine, and validate it both structurally, against an independent UPPAAL timed-automata model, and empirically, via a Python discrete-event simulation against a background (idle-time) aperiodic scheduler. The simulation shows that DSS substantially lowers mean aperiodic response time by preempting periodic execution on its own EDF deadline; that responsiveness is governed chiefly by the server period T_s rather than utilisation C_s/T_s alone; that the response-time distribution turns multi-modal once C_s approaches typical request execution times; and that periodic deadlines are met even under aperiodic overload, at the cost of aperiodic responsiveness.**

***Index Terms*—real-time systems, aperiodic scheduling, Earliest Deadline First, Dynamic Sporadic Server, schedulability analysis, UPPAAL, discrete-event simulation**

I. Introduction

A. Motivation

Real-time systems are increasingly required to support a mixture of workloads within a single processor: periodic activities such as control loops and sensor sampling, and aperiodic or sporadic activities such as operator inputs, fault handling, or event-driven sensor updates. Periodic tasks are typically associated with hard deadlines, where a single missed deadline can compromise system safety or correctness. Aperiodic tasks, by contrast, usually carry soft deadlines: timely service is desirable for system responsiveness, but an occasional delay does not lead to catastrophic failure. The scheduling problem is therefore not merely to guarantee the hard periodic task set, but to do so while minimising the average response time of the aperiodic workload, since a real-time system that is technically correct but unresponsive to operator or sensor events is of limited practical value.

B. Real-Time Scheduling Background

The mechanisms used to schedule periodic tasks fall broadly into two families: fixed-priority policies, such as Rate Monotonic (RM), where a task's priority is static and derived from its period, and dynamic-priority policies, such as Earliest Deadline First (EDF), where priority is recomputed continuously from each task's current absolute deadline. EDF is of particular interest for its schedulability optimality among preemptive uniprocessor algorithms (see Section III). This optimality, however, complicates the design of mechanisms for aperiodic service, since most early aperiodic-service algorithms were developed for the simpler, static priority ordering of fixed-priority systems and do not transfer directly to a deadline-driven environment.

C. Aperiodic Service in Fixed-Priority Systems

The simplest treatment of aperiodic tasks is *background scheduling*, in which aperiodic requests execute only in the idle intervals left by periodic tasks. This approach imposes no risk to the periodic guarantee but produces poor average response times whenever the periodic load is moderate to high, since aperiodic requests may be starved indefinitely behind periodic execution.

The *Polling Server* (PS) [1] improves on this by reserving a fixed capacity C_s within every period T_s, scheduled with a priority appropriate to T_s, for the exclusive use of aperiodic requests. The weakness of PS is that this capacity is only available at the instant the period begins: if no aperiodic request is pending at that instant, the capacity is discarded, even if a request arrives moments later. This idling behaviour makes PS *bandwidth-wasting* rather than *bandwidth-preserving*.

The *Deferrable Server* (DS) [1] addresses this specific weakness by allowing the server to retain its capacity throughout the period instead of discarding it immediately, so that a late-arriving aperiodic request can still be served at the server's priority. This preserves the server's responsiveness but introduces a more subtle problem, discussed in Section II: capacity deferral can lead to worst-case "double hit" behaviour that inflates interference on lower-priority periodic tasks beyond what standard RM analysis assumes.

The *Sporadic Server* (SS) [2] was proposed specifically to eliminate this anomaly while retaining capacity-preservation. Rather than replenishing capacity unconditionally at fixed period boundaries, SS ties each replenishment to the instant at which the corresponding capacity was *consumed*, delayed by exactly one server period T_s. This rule guarantees that the server's interference on the periodic task set never exceeds what a periodic task of the same period and capacity would cause, restoring exact RM schedulability analysis without the pessimistic correction required by DS. SS is, however, formulated entirely in terms of fixed priority levels and active priority tracking, which has no direct counterpart in

EDF, where a task's "priority" (its absolute deadline) changes continuously as time advances.

D. Aperiodic Service in EDF Systems

For EDF-scheduled systems, several dynamic-priority counterparts to these bandwidth-preserving servers have since been proposed. The *Total Bandwidth Server* (TBS) [3] assigns each arriving aperiodic request a deadline computed from the server's reserved utilisation and the deadline of the previous request (see Section III), guaranteeing that the server never consumes more than its allotted bandwidth over any interval. The *Constant Bandwidth Server* (CBS) [4] refines this further by postponing the server's deadline whenever its budget is exhausted before the next replenishment, which provides strong temporal isolation between the server and the rest of the task set even in the presence of execution-time overruns.

The *Dynamic Sporadic Server* (DSS) [3], the subject of this work, takes a different route: rather than computing deadlines from a bandwidth formula, it carries the SS replenishment rule itself over into EDF, reformulating "replenish C_s at T_s after consumption" in terms of deadlines rather than priority levels. Compared to TBS and CBS, DSS retains a closer structural relationship to its fixed-priority ancestor, which makes its behaviour and schedulability argument easier to relate directly to the well-understood SS analysis, at the cost of being somewhat less flexible in handling budget overruns than CBS's deadline-postponement mechanism. Compared to background and polling service, DSS is bandwidth-preserving by construction and therefore offers substantially better average response time for the same guaranteed periodic schedule.

II. Problem Statement

The central problem addressed by the DSS algorithm is to optimise the average response time of soft aperiodic requests under EDF, while guaranteeing that the hard periodic task set remains exactly as schedulable as it would be without the server present — that is, the server must behave, from the periodic tasks' perspective, indistinguishably from an ordinary periodic task of utilisation $U_s = C_s/T_s$.

This requirement is non-trivial precisely because EDF assigns priority through absolute deadlines rather than static levels. A naïve dynamic-priority server that simply restores its full capacity C_s and reassigns $d = t + T_s$ at every period boundary, irrespective of whether or when it actually executed, reproduces the same double-hit anomaly discussed above: it can demand its budget twice within an interval shorter than T_s. Under EDF, however, this excess demand manifests not as the priority-inversion problem it caused under fixed priorities, but as an unaccounted increase in the server's effective utilisation within a bounded interval, which can cause periodic deadlines to be missed despite $U \leq 1$.

The challenge, therefore, is to define a replenishment rule for C_s that:

1) preserves the server's capacity from the start of its period until an aperiodic request actually arrives, so that responsiveness is not sacrificed to idling, as in PS;
2) replenishes capacity consumed during a given execution burst no earlier than exactly one server period after that burst began, so that the server's cumulative demand over any interval of length T_s never exceeds C_s, avoiding the DS anomaly; and
3) expresses this rule purely in terms of deadlines and burst start instants, since EDF has no notion of a static "active priority level" to track, unlike the original SS formulation.

A further subtlety is the interaction between the server's own deadline and the deadlines of the aperiodic requests it serves: since EDF priority follows directly from deadline, the rule used to assign and update the server's deadline at execution and replenishment time *is* the rule that determines temporal isolation, and must be proven correct independently of any particular periodic task set, in the same way the original SS proof is independent of the specific RM priority assignment used. Establishing this replenishment rule, and showing that it preserves both responsiveness and the periodic schedulability guarantee, is the technical core of the DSS algorithm and the focus of the remainder of this work. Section III formalises this rule; Section IV presents it as an executable, burst-level algorithm.

III. Mathematical Foundations

A. Necessary Mathematical Basics

For a set of n independent periodic tasks τ_i, each with worst-case execution time C_i and period T_i (implicit deadline $D_i = T_i$), the total processor utilisation is

$$U_p = \sum_{i=1}^{n} \frac{C_i}{T_i}. \tag{1}$$

EDF is optimal among preemptive uniprocessor scheduling algorithms in the sense that such a task set is feasible under EDF if and only if $U_p \leq 1$. This bound is exact, unlike the sufficient-only Liu & Layland bound for RM, and it is the property that a bandwidth-preserving aperiodic server must not violate: the server is admitted into the system as an additional load of utilisation $U_s = C_s/T_s$, and the periodic set together with the server remains feasible only if $U_p + U_s \leq 1$ — provided the server's demand is itself bounded correctly, which is exactly what the replenishment rule below must guarantee.

B. Deadline Assignment and Replenishment Rule

1) Deadline Assignment on Activation: In an EDF environment, the server is treated as a periodic task with a dynamic deadline. When the server transitions from IDLE to READY at time t — i.e., an aperiodic request causes it to become active while it holds no current deadline — it is assigned an absolute deadline d_s such that

$$d_s = t + T_s. \tag{2}$$

This deadline is used by EDF exactly as any periodic task's deadline would be, and it remains fixed for the duration of the server's active period, regardless of how many times it is preempted and resumed within that period (see Section IV).

2) Burst-Level Consumption and Replenishment: Rather than tracking replenishment per infinitesimal unit of execution, DSS tracks it per *execution burst*: a maximal interval during which the server runs uninterrupted at its assigned deadline. Let burst k begin at t^k_{enter} and end at t^k_{exit} — due to preemption by a shorter-deadline periodic task, completion of the pending request, or exhaustion of c_s. The capacity consumed during that burst is

$$\epsilon_k = t^k_{\text{exit}} - t^k_{\text{enter}}, \tag{3}$$

and it is scheduled for replenishment at

$$t^k_r = t^k_{\text{enter}} + T_s, \tag{4}$$

i.e., one server period after the *burst began* — never earlier, and independent of when within the burst any particular fraction of ϵ_k was actually executed. At t^k_r, the corresponding amount is returned to the server's remaining capacity,

$$c_s\big((t^k_r)^+\big) = c_s\big((t^k_r)^-\big) + \epsilon_k. \tag{5}$$

An active period with no preemptions consists of a single burst, so this reduces to one replenishment of the server's full consumption, T_s after activation — recovering the simple case implied by Equation 2. An active period interrupted by preemptions instead accumulates several smaller pending replenishments, one per burst, each anchored to its own t^k_{enter}.

This burst-driven anchoring — rather than a per-instant one — is the same rule used by the original fixed-priority Sporadic Server, and it still guarantees that the server's cumulative execution within any interval of length T_s never exceeds C_s: since the server consumes no capacity while idle prior to t^k_{enter}, the entire burst ϵ_k falls within the window $[t^k_{\text{enter}}, t^k_{\text{enter}} + T_s)$, and that capacity is unavailable for any further consumption until t^k_r closes that same window. This is the formal counterpart of requirement 2 in Section II, and is what distinguishes DSS from the unconditional per-period replenishment of DS. Requirement 1 (capacity is not lost while idle) follows directly from Equation 2: the server only consumes and assigns deadlines relative to actual request arrivals, never discarding c_s at a period boundary the way PS does.

3) Schedulability Analysis: The inclusion of a DSS does not compromise the optimality of the EDF scheduler. A set of periodic tasks with utilisation U_p and a DSS of utilisation $U_s = C_s/T_s$ is schedulable if and only if

$$U_p + U_s \leq 1, \tag{6}$$

with U_p as defined above. This follows from the EDF feasibility bound in Section III together with the demand bound established by Equations 4–5: since the server never demands more than C_s within any interval of length T_s, it is indistinguishable, from the periodic task set's perspective, from an ordinary periodic task of utilisation U_s, and the standard EDF utilisation test applies unchanged.

IV. The Dynamic Sporadic Server Algorithm

Section III derived the deadline-assignment and burst-anchored replenishment rules as continuous-time equations and proved that they bound the server's demand within C_s. This section restates the same two rules as an explicit, event-driven state machine — the form actually used for implementation and for the UPPAAL and Python models of Sections VI and VII. The operation of the DSS is governed by a state machine consisting of three states: IDLE, READY, and EXE. In addition to the state itself, the server maintains a remaining capacity $c_s(t)$, an active deadline d_s (used for EDF priority comparisons against periodic tasks), and a replenishment queue holding pending (t_r, ϵ) pairs.

A. State Transitions

- **IDLE → READY**: Occurs either (i) when an aperiodic request enters the system while $c_s(t) > 0$, or (ii) when a queued replenishment fires and there is a backlogged request waiting on capacity that was previously exhausted. In either case, this transition marks the start of a new *active period*: if the server was fully idle beforehand (no active deadline held over from a previous active period), its deadline is set to

  $$d_s = t + T_s, \tag{7}$$

  matching Equation 2. This deadline is used purely for EDF priority ordering against periodic tasks and does not change again until the server returns to IDLE.
- **READY → EXE**: Occurs when the server holds the highest priority (shortest deadline) among ready tasks and a pending aperiodic request is available to run. The current time is recorded as the start of a new execution burst, $t_{\text{enter}} \leftarrow t$.
- **EXE → READY**: Occurs if the server is preempted by a periodic task with a shorter deadline before its current burst completes or its capacity is exhausted. Let $\epsilon = c_s(t_{\text{enter}}) - c_s(t)$ be the capacity consumed during the burst that just ended. A replenishment is scheduled:

  $$t_r = t_{\text{enter}} + T_s, \qquad \text{amount } \epsilon. \tag{8}$$

  The server re-enters READY still holding d_s and any remaining capacity, awaiting its next opportunity to execute.
- **EXE → IDLE**: Occurs if the pending aperiodic request completes, or if c_s is exhausted before the request completes. As in the preemption case, a replenishment of the capacity consumed during the final burst is scheduled per Equation 8, anchored to that burst's own t_{enter} — *not* to the original start of the active period. If the server ran an entire active period uninterrupted, this reduces to a single replenishment of the full amount consumed, scheduled at $t_{\text{enter}} + T_s$.

V. Runtime and Overhead Analysis

The computational overhead of the DSS algorithm primarily resides in the management of the replenishment queue and the deadline assignment logic.

A. Algorithm Complexity

The selection of the highest-priority task in an EDF system with N tasks typically requires $O(\log N)$ with a priority queue or $O(N)$ with a simple array. In our implementation, the `select_task` function performs a linear scan, resulting in $O(N)$ complexity per scheduling point.

The replenishment rule requires $O(1)$ time to schedule a new replenishment entry (appending to the circular buffer) and $O(1)$ time to process a replenishment when its timer expires. Because replenishments are burst-anchored rather than instant-anchored, the number of pending entries at any time is bounded by the number of preemptions the server experiences during a single active period, rather than by its capacity granularity — in practice a small, bounded number, so the queue's memory overhead remains modest.

VI. UPPAAL Model

To validate the state-machine description of Section IV independently of any particular implementation language, the DSS was additionally modelled in UPPAAL as a network of timed automata communicating over broadcast channels. Two `PeriodicTask` instances and one `DSS` instance realise the periodic task set and the server, respectively, each cycling through IDLE, READY, and ACTIVE locations mirroring the IDLE/READY/EXE states of Section IV. A `CPU` process tracks processor occupancy, a `Source` process emits aperiodic `request` events, and a `Ticker` process advances global time in fixed one-unit steps, invoking the EDF task-selection logic and broadcasting `reschedule` to keep every process synchronised.

The scheduling and replenishment logic itself is kept as plain, tick-driven code in the model's global declarations rather than as automaton structure: `select_task()` performs the EDF comparison over relative deadlines, and `update_dss()` advances a circular buffer of pending (t_r, ϵ) replenishment pairs exactly as described in Section IV. This keeps the automaton structure small while letting the bookkeeping that matters for correctness be inspected as ordinary code alongside the model.

UPPAAL's verifier is, however, a finite-state model checker, whereas the DSS modelled here is inherently time-driven: its deadline counters, budget, and replenishment queue evolve over an effectively unbounded number of discrete ticks as the simulation horizon grows. Exhaustive reachability and safety queries therefore cannot yield a sustainable, tool-independent correctness guarantee the way they would for a naturally finite-state protocol. Accordingly, the UPPAAL model serves here as an executable cross-check of the state-machine formalisation against a second, independent tool, rather than as a proof of correctness; empirical validation is carried out instead through the discrete-event simulation of the following section.

VII. Simulation

To complement the formal model of Section VI with a more direct, statistical view of the algorithm's benefit, the DSS was implemented in Python together with a simple EDF scheduler and, as a baseline, a purely background ("idle-time") aperiodic scheduler that assigns aperiodic requests the lowest possible EDF priority. Development of this simulation was assisted by AI tools [5], [6]; the resulting code was then used to run a range of DSS configurations, with the observed scheduling behaviour analysed further below.

A. Simulation Setup

All simulations share the same periodic task set, $\{(T_i, C_i)\} = \{(10, 2), (20, 4), (50, 6)\}$, giving $U_p = 0.52$ and therefore an admissible server utilisation of $U_s \leq 0.48$, and the same pseudo-random seed, so that the sequence of aperiodic arrival times and execution times is identical across runs. The one exception is Section VII-F, where the aperiodic arrival probability itself is the varied parameter: the seed is unchanged, but a higher arrival probability naturally produces additional requests beyond those seen at the baseline rate. Otherwise, only the DSS budget C_s and period T_s are varied between runs.

B. Preemption and Response Time

Fig. 1 and Fig. 2 show the same 60 ms window of execution under the background scheduler and under DSS ($C_s = 2, T_s = 5$), respectively. Under the background scheduler, aperiodic execution is confined entirely to intervals in which no periodic job is ready, so the first aperiodic burst is delayed until $t \approx 14$ ms even though it arrives earlier.

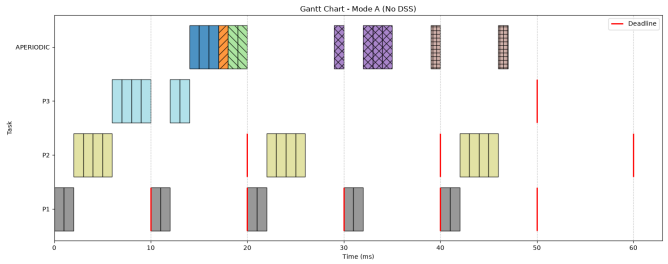

Fig. 1. Background (idle-time) scheduler.

Under DSS, the server competes on its own EDF deadline like any other task, so it preempts periodic execution directly from $t = 1$ and aperiodic requests are served with markedly lower response time — while the periodic tasks still meet every deadline in both modes.

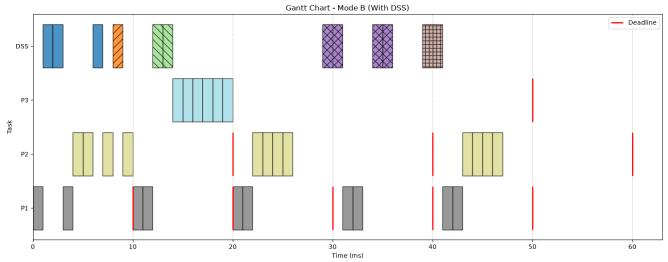

Fig. 2. DSS, $C_s = 2$, $T_s = 5$.

C. Budget/Period Granularity at Fixed Utilisation

Fig. 3 repeats the same window with $C_s = 4$, $T_s = 10$ — both parameters doubled, so $U_s = 0.4$ is unchanged from Fig. 2. The two runs nonetheless differ: with the larger budget, a burst that would previously have exhausted the server's capacity and forced a wait for replenishment instead completes in a single, uninterrupted execution. Equal U_s does not imply equal scheduling behaviour — C_s and T_s individually determine the granularity at which the server's capacity becomes available.

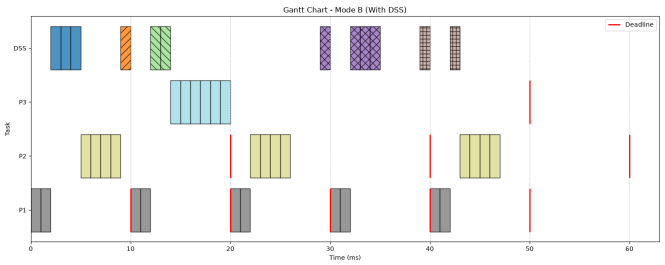

Fig. 3. DSS, $C_s = 4$, $T_s = 10$ (same $U_s = 0.4$ as Fig. 2).

D. Effect of Server Period on Response Time

Fig. 4–6 compare response-time distributions, over a full 50 s run, for $(C_s, T_s) \in \{(6, 13), (12, 25), (24, 50)\}$.

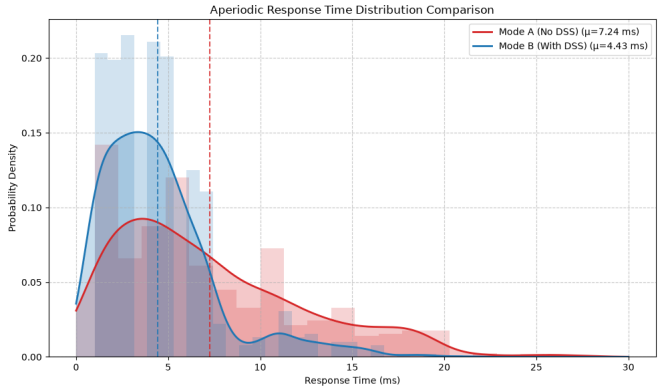

Fig. 4. $C_s = 6$, $T_s = 13$.

Here $12/25 = 0.48$ is the maximum utilisation the server can be given without violating $U_p + U_s \leq 1$, so $(24, 50)$ carries the same nominal utilisation, while $(6, 13)$ is very slightly below it — 13 is the nearest integer period above the unattainable 12.5, so rounding actually reduces the achievable U_s here. However, mean aperiodic response time decreases monotonically as T_s shrinks.

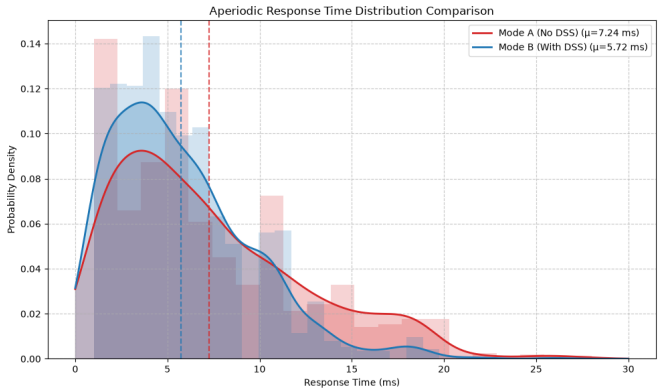

Fig. 5. $C_s = 12$, $T_s = 25$.

The $(24, 50)$ case is degenerate: since the server's EDF deadline is set to $t + T_s$ on activation, a period as large as $T_s = 50$ gives the DSS a deadline no more urgent than the periodic tasks', so it is essentially always outranked by them and behaves indistinguishably from background service despite holding a reserved budget.

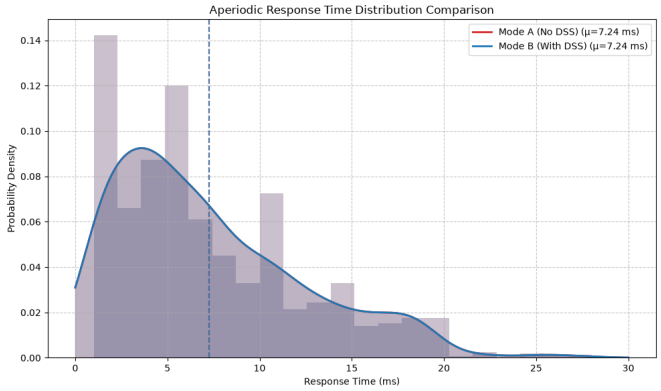

Fig. 6. $C_s = 24$, $T_s = 50$.

This confirms that, for fixed U_s, it is specifically the period T_s — through its direct role in the server's EDF priority — that governs responsiveness, and this benefit holds only while the aperiodic load remains modest relative to U_s (see Section VII-F).

E. Multi-Modal Response Time Distributions

A further effect appears once the budget C_s becomes comparable to the mean aperiodic execution time (uniformly distributed over $[1, 5]$ ms, mean 3 ms): the response-time distribution becomes multi-modal. In Fig. 7 ($C_s = 3$, $T_s = 7$), one peak groups requests with execution time ≤ 3 ms, which complete within a single burst; a second peak appears roughly one server period later, corresponding to requests of 4–5 ms that need one additional replenishment before they can finish.

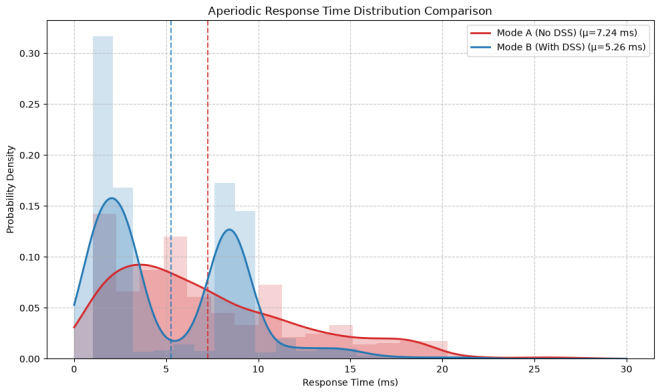

Fig. 7. $C_s = 3$, $T_s = 7$.

In Fig. 8 ($C_s = 2$, $T_s = 5$), the same effect produces three peaks, since only 2 ms of execution fit in a single burst: requests of 1–2 ms finish immediately, 3–4 ms requests are delayed by roughly one period, and 5 ms requests by roughly two periods.

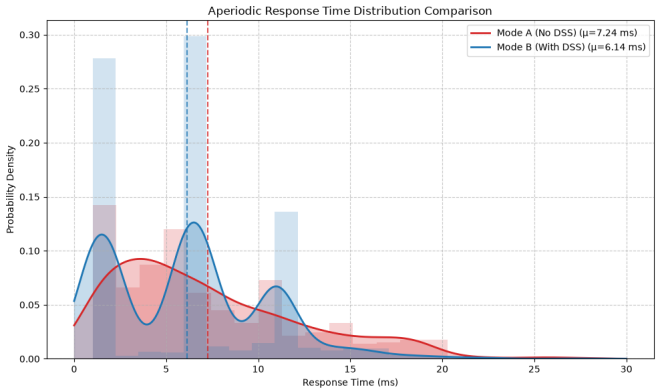

Fig. 8. $C_s = 2$, $T_s = 5$.

F. Effect of Aperiodic Load

Finally, $C_s = 12$, $T_s = 25$ is held fixed while the aperiodic arrival probability is increased from the baseline 0.025 (Fig. 5, mean response 5.72 ms under DSS versus 7.24 ms under background service) to 0.05 (Fig. 9, 7.99 ms vs. 8.17 ms) and 0.10 (Fig. 10, 13.90 ms vs. 12.97 ms).

As the offered aperiodic load grows toward and past the server's reserved bandwidth $U_s = 0.48$, mean response time under DSS worsens and eventually exceeds that of the background scheduler, since a fixed budget and period bound how much aperiodic work can be absorbed per period regardless of demand.

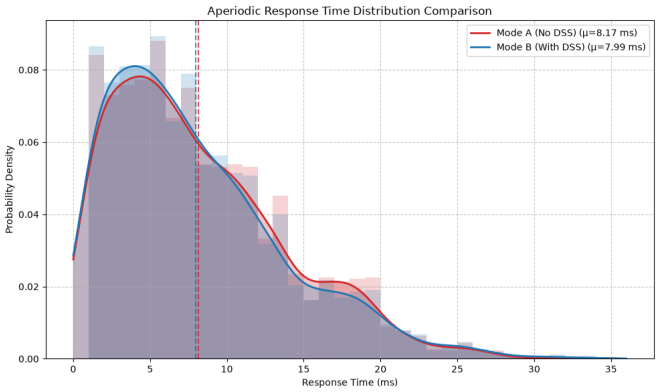

Fig. 9. Arrival probability 0.05 (2× baseline).

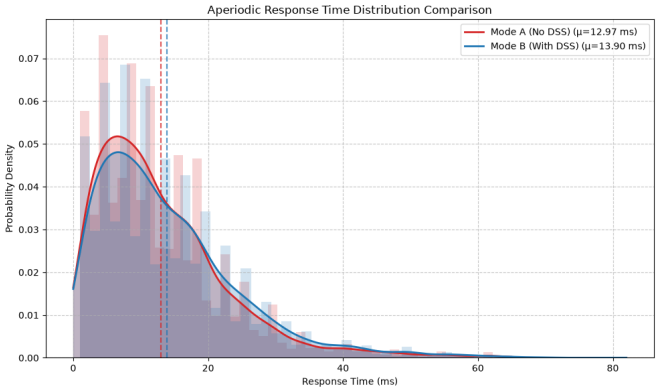

Fig. 10. Arrival probability 0.10 (4× baseline).

Crucially, this degradation is confined to the aperiodic side: consistent with the schedulability guarantee of Section III, the periodic task set continued to meet every deadline in all three runs, since the server's interference is capped by (C_s, T_s) independently of how many aperiodic requests are actually waiting. What changes under overload is only how much of that fixed bandwidth is available per request — once demand outpaces it, requests increasingly queue and remain unserved by the end of the simulation horizon.

VIII. Conclusion

The burst-anchored replenishment rule derived in Sections II–V and validated in Sections VI–VII yields the following results:

- **Bounded demand, low overhead.** The rule bounds the server's demand within any interval of length T_s by C_s, giving $O(N)$ per-scheduling-point and $O(1)$ per-replenishment overhead.
- **Preemption benefit.** By competing on its own EDF deadline, DSS reduces mean aperiodic response time relative to background scheduling, without causing any periodic deadline miss.

- **Granularity matters.** For a fixed utilisation $U_s = C_s/T_s$, the individual values of C_s and T_s still determine the granularity at which capacity is made available — equal utilisation does not imply equal scheduling behaviour.
- **Period governs responsiveness.** Because the server's EDF deadline is set to $t + T_s$ on activation, it is T_s — not U_s alone — that governs responsiveness; a large T_s can make DSS degenerate into background-equivalent behaviour even at maximal admissible utilisation.
- **Multi-modal response times.** Once C_s becomes comparable to individual request execution times, replenishment quantisation splits the response-time distribution into multiple modes.
- **Overload behaviour.** Under sustained aperiodic overload, DSS's mean response time can exceed that of the background scheduler, even though the periodic guarantee is never at risk.

Taken together, these results confirm that DSS delivers substantially improved aperiodic responsiveness at no cost to periodic schedulability, subject to a careful choice of T_s and to the aperiodic load remaining within U_s. The evaluation was limited to a single periodic task set, a single aperiodic workload distribution, and a synthetic arrival process; future work should broaden the parameter sweep, compare DSS empirically against TBS and CBS under identical conditions, and evaluate the algorithm on a real-time kernel to measure implementation overhead beyond the asymptotic analysis of Section V.

REFERENCES

[1] J. K. Strosnider, J. P. Lehoczky, and L. Sha, "The deferrable server algorithm for enhanced aperiodic responsiveness in hard real-time environments," *IEEE Transactions on Computers*, vol. 44, pp. 73–91, 1995. [Online]. Available: https://ieeexplore.ieee.org/document/368008

[2] B. Sprunt, L. Sha, and J. Lehoczky, "Aperiodic task scheduling for hard-real-time systems," *Real-Time Systems 1989 1:1*, vol. 1, pp. 27–60, 1989. [Online]. Available: https://link.springer.com/article/10.1007/BF02341920

[3] M. Spuri and G. Buttazzo, "Scheduling aperiodic tasks in dynamic priority systems," *Real-Time Systems*, vol. 10, pp. 179–210, 3 1996. [Online]. Available: http://link.springer.com/10.1007/BF00360340

[4] L. Abeni and G. Buttazzo, "Integrating multimedia applications in hard real-time systems," *Proceedings - Real-Time Systems Symposium*, pp. 4–13, 1998. [Online]. Available: https://ieeexplore.ieee.org/document/739726

[5] Anthropic, "Claude [large language model]," 2026, used during software development, debugging, code generation and text drafting. Accessed: 2026-07-10. [Online]. Available: https://claude.ai

[6] OpenAI, "ChatGPT (gpt-5.5) [large language model]," 2026, used during software development, debugging, code generation and text drafting. Accessed: 2026-07-08. [Online]. Available: https://chatgpt.com